

Webp Me Daddy

A reusable Codex skill for turning website images into recipe-driven, lintable WebP assets with structured metadata, accessibility-safe alt text, and paste-ready snippets.

Shave bytes. Keep vibes. Retire hero-final-final.jpg with dignity.

WebP con version	Semantic recipes	Contain logos	Usage overrides	V2 manifests	Audit reports	Cleanup mode	Lint-ready output
------------------	------------------	---------------	-----------------	--------------	---------------	--------------	-------------------

What The Skill Does

- Convert **.jpg** and **.png** files into optimized **.webp** assets.
- Drive the workflow with semantic recipes like **hero-banner**, **blog-cover**, **profile-avatar**, **card-thumbnail**, **poster**, **story-cover**, **logo-lockup**, and **logo-grid**.
- Shift the crop toward faces, text, or off-center subjects with focus presets or exact normalized coordinates.
- Preserve transparent lockups and wordmarks with contain-mode outputs instead of forcing a crop.
- Keep small source images at native size by default so recipe targets do not accidentally upscale them.
- Normalize file names into SEO-friendly, hyphen-case slugs.
- Generate suggested **alt**, **title**, and **caption** metadata.
- Store named usage-level overrides when the same asset needs different metadata on different pages.
- Create paste-ready **HTML** and **React** snippets inside a versioned v2 sidecar/manifest contract.
- Create framework-ready **Next.js** and **Astro** snippets alongside the generic targets.
- Rebuild page-specific snippets from a sidecar and named usage override without regenerating the image.
- Lint manifests for blocking accessibility issues and warning-level performance issues.
- Audit a whole codebase for live PNG/JPEG references, animated assets, shared-context assets, unused public assets, and markup gaps.
- Emit ready-to-paste JSX autofix patches and file-specific codemod replacements when the audit can safely infer missing image attributes.
- Hand GIF and animated WebP assets to the transparent loop optimizer from the same top-level CLI.
- Preview or delete unused public assets with a cleanup command that reuses the audit rules.

Key Features

Feature	What it enables
Prepare command	Convert one source image into a finished WebP asset with metadata and recipe-aware defaults.
Batch command	Process a whole folder with explicit confirmation and shared recipe settings.
Lint command	Validate the manifest contract and catch missing alt or broken accessibility usage.
Semantic recipes	Map user intent to layout, widths, loading hints, and snippet defaults.

Contain fit mode	Preserve transparent or edge-sensitive artwork inside fixed output dimensions.
Logo-grid recipe	Handle sponsor badges and partner logos with a contain-first tile recipe.
No-upscale default	Protect small source images from becoming heavier or blurrier just to match a recipe target.
Accessibility enum	Handle decorative, logo, descriptive, and text-bearing images consistently.
Usage overrides	Keep one clean default alt while storing page-specific copy for reused assets.
V2 schema	Capture structured sidecars and manifests that can feed future tooling and a web app.
Snippets command	Rebuild page-specific markup from a sidecar and usage key without touching the source asset.
Audit command	Scan src/public usage, report live legacy formats, and emit JSX autofix plus file-specific codemod patches.
Animate command	Shell animated GIF/WebP inputs into the transparent loop optimizer without leaving the main CLI.
Cleanup command	Preview or prune unused public assets after an audit without inventing a second rule set.

Common Use Cases

- Prepare a homepage hero with the **hero-banner** recipe and eager-loading defaults.
- Turn a creator portrait into a **profile-avatar** WebP with cleaner naming and metadata.
- Create a **poster** asset for vertical promo art and off-center subjects.
- Generate a **card-thumbnail** for feeds, cards, and editorial promos.
- Generate a **story-cover** for short-form vertical placements.
- Preserve a partner logo or transparent brand lockup with **logo-lockup** and contain-mode sizing.
- Prepare a sponsor badge or creator-program tile with **logo-grid** and decorative-first defaults.
- Run a dry-run folder pass, then lint the manifest before shipping.
- Audit an existing site to find lingering PNGs, animated assets, shared-usage art, unused assets, or tags missing width, loading, or decoding hints.
- Use the sidecar-driven snippets helper to regenerate markup for one page placement after you refine a usage override.
- Send GIF or animated WebP sticker-style assets through the explicit animate handoff instead of forcing them through the still-image flow.
- Run a cleanup dry-run to preview which stale source files could be safely removed after optimization.
- Hand structured sidecars and manifests to later tools without re-parsing filenames.

Accessibility And SEO

- **Decorative mode** emits empty alt text plus presentation hints.
- **Logo mode** keeps brand-mark alt text concise instead of keyword-heavy.
- **Descriptive mode** handles the default informative case cleanly.
- **Text-bearing mode** lets the final alt text reference exact copy when it matters.
- **Usage overrides** keep repeated assets honest when homepage, deck, and article contexts need different copy.

- **Contain mode** protects transparent logos, wordmarks, and lockups from destructive crops.
- **No-upscale defaults** keep tiny images honest unless the user explicitly opts into enlargement.
- Snippets escape attributes safely so quotes and special characters do not break markup.
- Audit autofix patches and codemod entries give you a fast starting point for width, height, loading, and decoding cleanup in JSX.

Workflow Summary

1	Confirm the semantic recipe and accessibility mode.
2	If the subject is off-center, choose a focus preset or set exact focus coordinates.
3	If the artwork is a logo or lockup, use contain mode to preserve its full composition.
4	Use dry-run first for large or risky folder operations.
5	Run prepare or batch and review the v2 sidecar or manifest.
6	Run lint when you want a quality gate before shipping.
7	Use a usage override when one asset needs different metadata in different placements.
8	Use the snippets helper when you want fresh page-specific markup from a sidecar and usage key.
9	Run audit when you want a codebase-level report on image usage, animated assets, and markup gaps.
10	Run animate for GIF or animated WebP assets that belong in the loop workflow.
11	Run cleanup in dry-run mode, then confirm with deletion mode when the candidate list looks right.
12	Paste or adapt the snippets into the site codebase.

Example Commands

Prepare one hero:

```
python C:/Users/Alex/.codex/skills/webp-me-daddy/scripts/webp_me_daddy.py prepare public/source.jpg --recipe hero-banner --usage-key home.hero --usage-alt \"Homepage hero illustration\" --public-root public --write-sidecar
```

Poster with top focus:

```
python C:/Users/Alex/.codex/skills/webp-me-daddy/scripts/webp_me_daddy.py prepare public/poster.jpg --recipe poster --focus-preset top --public-root public --write-sidecar
```

Logo lockup with contain mode:

```
python C:/Users/Alex/.codex/skills/webp-me-daddy/scripts/webp_me_daddy.py prepare public/logo.png --recipe logo-lockup --accessibility-mode logo --public-root public --write-sidecar
```

Logo grid tile:

```
python C:/Users/Alex/.codex/skills/webp-me-daddy/scripts/webp_me_daddy.py prepare public/badge.png --recipe logo-grid --public-root public --write-sidecar
```

Batch preview:

```
python C:/Users/Alex/.codex/skills/webp-me-daddy/scripts/webp_me_daddy.py batch public --recipe b
log-cover --dry-run --manifest image-manifest.json --seo-context \"Brin Shadewater website\"
```

Lint manifest:

```
python C:/Users/Alex/.codex/skills/webp-me-daddy/scripts/webp_me_daddy.py lint image-manifest.js
on
```

Audit project:

```
python C:/Users/Alex/.codex/skills/webp-me-daddy/scripts/webp_me_daddy.py audit C:/path/to/projec
t --json image-audit.json
```

Snippet helper:

```
python C:/Users/Alex/.codex/skills/webp-me-daddy/scripts/webp_me_daddy.py snippets public/hero-so
urce.json --usage-key home.hero --target react
```

Animated handoff:

```
python C:/Users/Alex/.codex/skills/webp-me-daddy/scripts/webp_me_daddy.py animate public/spin.web
p public/spin-clean.webp --size 220 --bridge-frames 8
```

Cleanup preview:

```
python C:/Users/Alex/.codex/skills/webp-me-daddy/scripts/webp_me_daddy.py cleanup C:/path/to/proj
ect --dry-run --json cleanup-report.json
```

Outputs

- Optimized **.webp** files
- Optional responsive variants like **image-480w.webp**
- Optional per-image v2 JSON sidecars
- Optional v2 manifest JSON for prepare or batch runs
- Optional audit JSON for codebase-level image reviews plus JSX autofix and codemod suggestions
- Optional cleanup JSON for candidate deletions and reclaimed bytes
- Paste-ready HTML, React, Next.js, and Astro snippets plus lint metadata
- On-demand snippet exports for one named usage key

Validation

- Run **test_prepare_image.py** after meaningful script changes.
- Run **webp_me_daddy.py audit <project-root>** and **cleanup <project-root> --dry-run** against a real site to verify the reporting output.
- Smoke-test the **snippets** and **animate** commands so the helper and loop handoff stay aligned with the main CLI.
- Run the skill validator and visually review the explainer PDF before sharing it.